



Formation Spring

1

Accéder aux données

Accès aux données et JDBC avec Spring

2

- ✓ Problématiques
- ✓ Solutions apportées par le Spring
- ✓ Présentation et usage du namespace JDBC
- ✓ La DataSource
- ✓ Le JdbcTemplate
- ✓ Les différents Mapper

Intégration de Spring

Problématiques des accès en base de données

- Les accès aux bases de données requièrent une gestion des ressources souvent délicates
 - L'application doit systématiquement
 - ☐ Etablir une connexion
 - ☐ Initier une transaction
 - ☐ Effectuer un commit ou rollback de la transaction
 - ☐ Fermer la connexion
- Cette gestion des ressources vient polluer le code applicatif
- Elles n'apportent aucune valeur ajoutée métier

Intégration de Spring

Problématiques des accès en base de données

- La solution consiste à apporter une couche d'abstraction
 - Automatiser la gestion des connections
 - ❑ Acquisitions/libérations des connections automatique
 - ❑ Eviter les fuites mémoires
 - Faciliter la gestion des transactions
 - ❑ Configuration des transactions externalisées
 - ❑ Utilisation d'un gestionnaire de transaction (pas forcément Java EE)
 - Simplifier la gestion des erreurs
- Considérer les accès aux ressources comme de bas niveau

Intégration de Spring

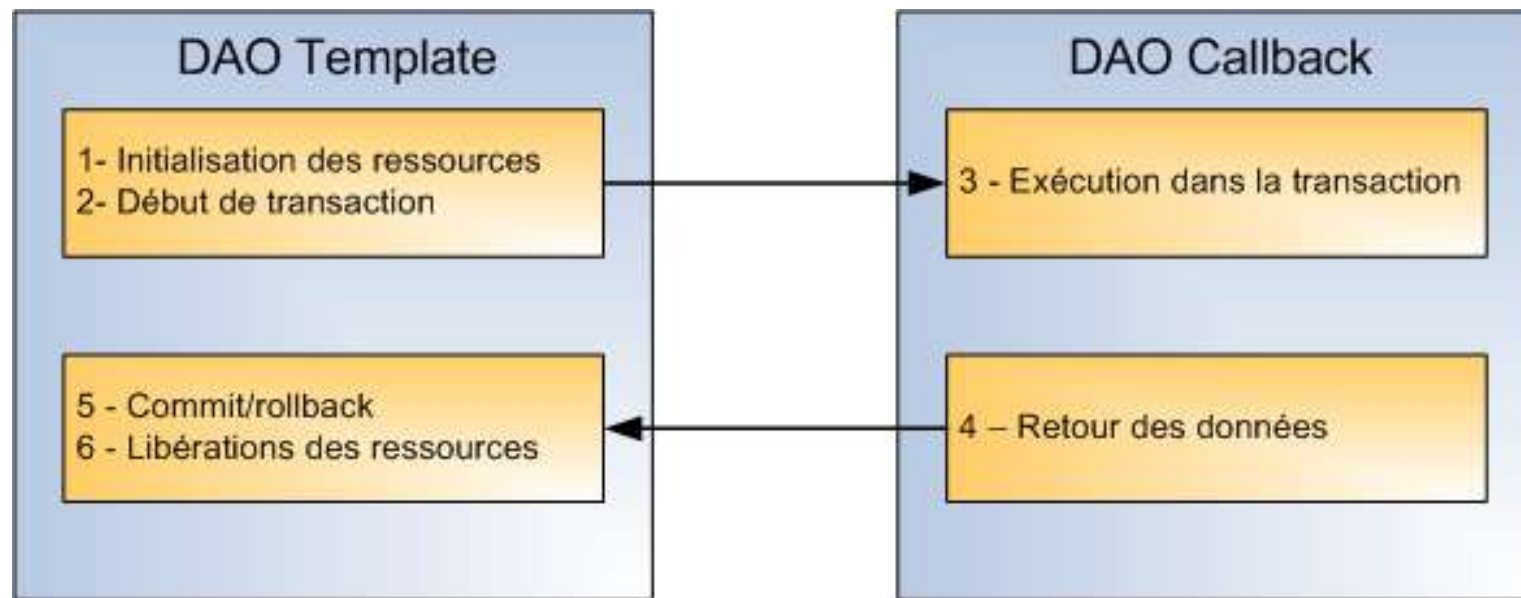
Solutions apportées (1/2)

- Une intégration avec les outils de persistance suivants :
 - JDBC et Hibernate
 - Ibatis, EJB, JPA.
- Utilisation du design pattern « Template Method » pour effectuer les accès des données
 - Fournit un « patron » spécifique pour chaque technologie.
 - Masque les détails techniques au développeur.
- Définition d'une hiérarchie d'exception générique
 - Encapsule les exceptions spécifiques des outils de persistance.
- Les transactions sont configurables par XML/annotations et déléguées automatiquement au gestionnaire de transactions

Intégration de Spring

Solutions apportées (2/2)

➤ Synoptique du pattern « Template Method »



Hiérarchie d'exceptions d'accès aux données

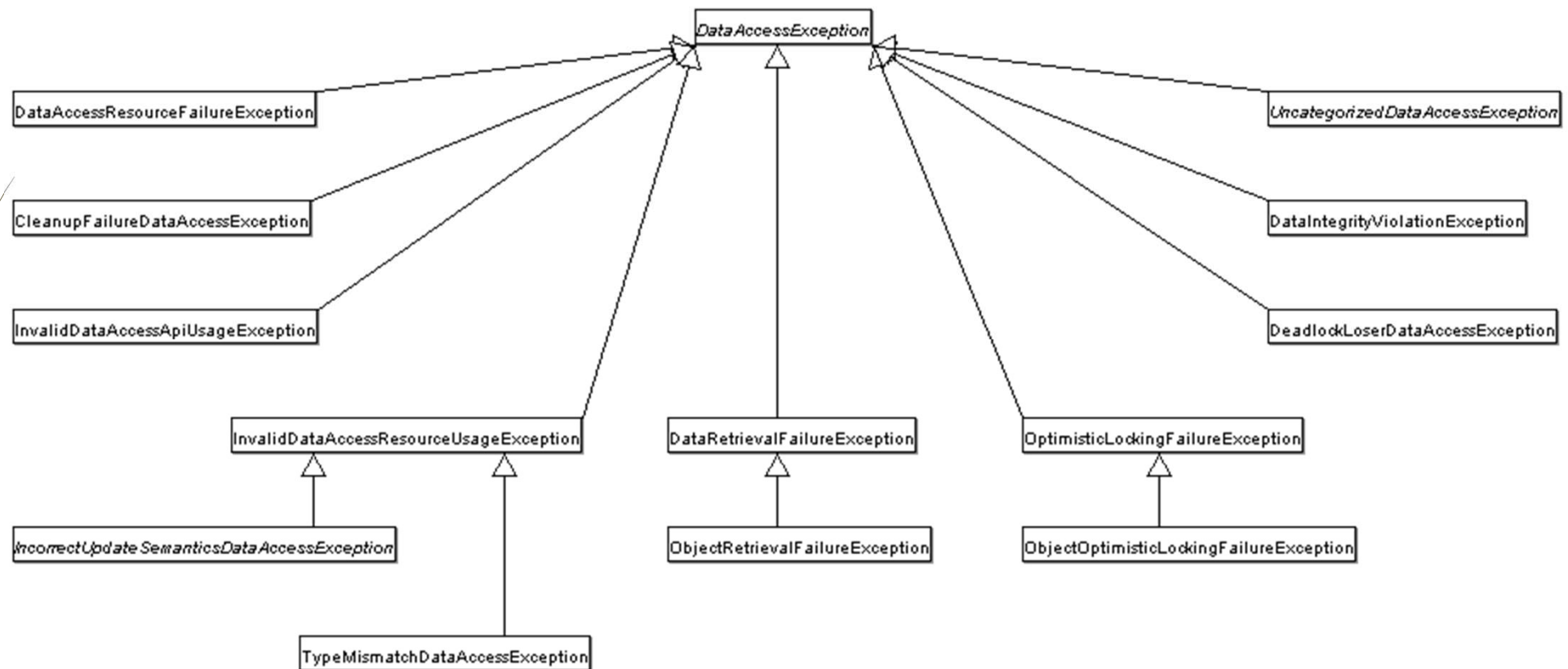
- Les exceptions de type « checked »
 - Oblige les développeurs à gérer ces exceptions.
 - Les développeurs ne savent pas quoi en faire.
 - Considéré comme mauvaise pratique
 - ❑ Les méthodes de niveau intermédiaires doivent déclarer les exceptions des couches inférieures.
 - ❑ Couplage fort avec les briques techniques de couches inférieures.
- Les exceptions de type « unchecked »
 - Les exceptions sont levées et interceptées aux endroits les mieux indiqués.
 - Les méthodes intermédiaires n'ont pas à gérer les exceptions.
- Spring lève des exceptions de type « Runtime » (unchecked)

Hiérarchie d'exceptions d'accès aux données

- L'objectif est de pouvoir s'abstraire des exceptions spécifiques aux implémentations
 - SQLException ou HibernateException.
- Spring fournit la hiérarchie « `DataAccessException` »
 - Type « Runtime » unchecked.
 - Encapsule les types d'exceptions quelque soit la technologie.
- Quelques exceptions
 - `CleanupFailureDataAccessException`
 - ❑ Une exception a été levée pendant la libération de ressources (exemple méthode `close()` sur une connexion)
 - `DataRetrievalFailureException`
 - ❑ Une erreur s'est produite lors d'une requête de sélection
 - `DeadlockLoserDataAccessException`
 - ❑ Problème d'accès concurrents, processus bloqué par un lock

Hiérarchie d'exceptions d'accès aux données

➤ Diagramme UML de la hiérarchie d'exception Spring



Data source embarquée (1/2)

En XML via name space jdbc



- L'espace de nom JDBC apparait avec la version 3 de Spring
- Spring propose deux tags XML
 - **<jdbc:embedded-database>** - met à disposition d'une datasource embarquée.
 - **<jdbc:initialize-database>** - initialise une base de données.
- Adapté pour la réalisation de tests unitaires automatisés ou non
 - S'intègre avec les bases de données intégrées.
 - Supporte nativement les outils HSQL (par défaut), H2 et Derby.

Data source embarquée (2/2)

En XML via name space jdbc



➤ Création d'une source de données avec HSQL

```
<jdbc:embedded-database id="dataSource">
  <jdbc:script location="classpath:db/schema.sql" />
  <jdbc:script location="classpath:db/init-data.sql" />
</jdbc:embedded-database>
```

- Source de données est utilisable de façon habituelle.
- Pour changer de base de données, renseigner l'attribut « type ».

➤ Initialiser une base de données en indiquant les scripts SQL à exécuter

```
<jdbc:initialize-database data-source="dataSource">
  <jdbc:script location="classpath:db/init-db.sql" />
</jdbc:initialize-database>
```

- L'attribut « data-source » fait référence à une source de données quelconque.
- L'attribut « enabled » active/désactive l'initialisation (utilisable avec SpEL).

Data source embarquée En @Configuration



- Vous pouvez utiliser l'objet `org.springframework.jdbc.datasource.embedded.EmbeddedDatabaseBuilder` et `org.springframework.jdbc.datasource.embedded.EmbeddedDatabase`

```
@Bean
public DataSource dataSourceEmbarquee() {
    EmbeddedDatabaseBuilder builder = new EmbeddedDatabaseBuilder();
    EmbeddedDatabase db = builder
        .setType(EmbeddedDatabaseType.HSQL)
        .addScript("db/schema.sql")
        .addScript("db/init-data.sql")
        .build();
    return db;
}
```

La Data source (1/4)

- Objet qui sera lié
 - Au JdbcTemplate (Spring JDBC)
 - Au TransactionManager
 - A la SessionFactory , EntityManagerFactory (Spring Hibernate, JPA)
 - ... tout ce qui touche à l'accès aux données.
- L'implémentation de cet objet (son attribut class) aura un impact très important sur les performances de vos projets
- Toutes les implémentations doivent respecter l'interface standard `javax.sql.DataSource`

La Data source (2/4)

14

➤ org.apache.commons.dbcp.**BasicDataSource**

- Data source commons-dbc Apache (Java < 7)
- <https://commons.apache.org/proper/commons-dbc/>

➤ org.apache.commons.dbcp2.**BasicDataSource**

- Data source commons-dbc2 Apache (Java >= 7)

➤ org.springframework.jdbc.datasource.**DriverManagerDataSource**

- Data source du Spring. A ne pas utiliser en production.

➤ com.mchange.v2.c3p0.**ComboPooledDataSource**

- Data source ultra configurable (mais plus vraiment à jour)
- <https://www.mchange.com/projects/c3p0/>

➤ com.zaxxer.hikari.**HikariDataSource**

- Data source ultra configurable (successeur de C3p0)
- <https://github.com/brettwooldridge/HikariCP>

La Data source (3/4)



➤ Exemples en XML

- Ne pas oublier d'ajouter les dépendances nécessaires dans son Maven

```
<bean    id="dataSource"
        class="org.apache.commons.dbcp.BasicDataSource"
        destroy-method="close">
  <property name="driverClassName" value="${jdbc.driverClassName}"/>
  <property name="url" value="${jdbc.url}"/>
  <property name="username" value="${jdbc.username}"/>
  <property name="password" value="${jdbc.password}"/>
</bean>
```

- Rappel : name="url" => il existe une méthode setUrl dans la classe
- <https://javadoc.io/static/commons-dbcp/commons-dbcp/1.4/org/apache/commons/dbcp/BasicDataSource.html>

La Data source (3/4)



➤ Exemples en classe de configuration

```
@Autowired
private Environnement env;

@Bean(destroyMethod = "close")
public DataSource dataSource() {
    BasicDataSource resu = new org.apache.commons.dbcp2.BasicDataSource();
    resu.setDriverClassName(env.getProperty("db.driver"));
    resu.setUrl(env.getProperty("db.url"));
    resu.setUsername(env.getProperty("db.login"));
    resu.setPassword(env.getProperty("db.pwd"));
    return resu;
}
```

➤ <https://commons.apache.org/proper/commons-dbcp/apidocs/org/apache/commons/dbcp2/BasicDataSource.html>

La Data source (4/4)



➤ Projet Web

- Il n'est pas rare que la data source soit déclarée au niveau du serveur JEE
- Dans ce cas, il faut simplement brancher sa data source Spring sur la data source du serveur
 - ❑ Utilisation du name space jee via configuration XML

```
<jee:jndi-lookup  
    id="dataSource"  
    jndi-name="jdbc/NomDuPool"  
    resource-ref="true"  
    lookup-on-startup="true"  
    proxy-interface="javax.sql.DataSource"/>
```

La Data source (4/4)



- Dans le cas des classes de configuration

```
@Bean
public DataSource dataSource() throws NamingException {
    JndiTemplate jndi = new JndiTemplate();
    return jndi.lookup("java:comp/env/jdbc/netbankPool", DataSource.class);
}
```

- "jdbc/netbankPool" : nom de la data source

Les accès JDBC avec le JdbcTemplate

- Le Spring met à disposition l'objet **JdbcTemplate**.
- Il s'appuie sur une source de données
 - Lors de sa déclaration/instanciation en tant que bean, il devra donc être lié avec une DataSource
- Il est « thread safe » et donc réutilisable
 - On peut aussi le placer en tant qu'attribut sur la classe mère de ses DAOs

Les accès JDBC avec le JdbcTemplate

➤ Exemple d'utilisation

@Repository

```
public class SpringJdbcPersonDaoImpl implements IPersonDao {
```

@Autowired

```
private JdbcTemplate jdbcTemplate;
```

@Override

```
public int fetchPersonCount() {  
    String sql = "SELECT COUNT(1) FROM PERSON";  
    return this.jdbcTemplate.queryForInt(sql);  
}  
}
```

Les accès JDBC avec le JdbcTemplate

Les requêtes simples

- JdbcTemplate permet de faire des requêtes sur :
 - Des collections (Map, List, etc...).
 - Des objets Métiers.

```
public int fetchPersonCount() {  
    String sql = "SELECT COUNT(1) FROM PERSON";  
    return this.jdbcTemplate.queryForObject(sql, Integer.class);  
}
```

```
public int fetchPersonCountById(Integer pkId) {  
    String sql = "SELECT COUNT(1) FROM PERSON WHERE ID = ?";  
    return this.jdbcTemplate. queryForObject(sql, Integer.class, pkId);  
}
```

Les accès JDBC avec le JdbcTemplate

Les requêtes génériques (1/2)

- JdbcTemplate est capable de retourner chaque ligne d'un « ResultSet » sous forme d'une Map
 - Une seule ligne attendue – queryForMap
 - Plusieurs lignes attendues – queryForList
- Résultats sur un seul tuple

```
public Map<String, Object> fetchPersonById(Integer pkId) {  
    String sql = "SELECT * FROM PERSON WHERE ID = ?";  
    return this.jdbcTemplate.queryForMap(sql, pkId);  
}
```

- Résultat obtenu : {ID=1, FNAME=Robert, LNAME=Durand}
- Une Map de la forme d'un couple [Nom Colonne | Nom champ]

Les accès JDBC avec le JdbcTemplate

Les requêtes génériques (2/2)

➤ Résultats sur plusieurs tuples

```
public List<Map<String, Object>> fetchPersonLikeLastName(String lname) {  
    String sql = "SELECT * FROM PERSON WHERE LNAME like ?%";  
    return this.jdbcTemplate.queryForList(sql, lname);  
}
```

➤ Résultat obtenu

List {

0 – Map {ID=1, FNAME=Robert, LNAME=Durand}

1 – Map {ID=2, FNAME=Jean, LNAME=Durant}

}

Les accès JDBC avec le JdbcTemplate RowMapper (1/3)

- JdbcTemplate permet d'associer les données sous forme d'objets métiers
- JdbcTemplate s'appuie sur la technique du « callback »
- Ce mapping est équivalent à celui utilisé par les outils ORM (Hibernate)
- JdbcTemplate met à disposition l'interface « RowMapper » pour associer les données
 - Peut être utilisé aussi pour des requêtes uniques ou sur des ensembles.

```
public interface RowMapper<T> {  
    T mapRow(ResultSet rs, int rowNum) throws SQLException;  
}
```


Les accès JDBC avec le JdbcTemplate RowMapper (2/3)

➤ Résultat sur un seul tuple

```
public Person fetchPersonById(Integer pkId) {  
    String sql = "SELECT * FROM PERSON WHERE ID = ?";  
    return this.jdbcTemplate.queryForObject(sql, new PersonRowMapper(), pkId);  
}
```

```
public class PersonRowMapper implements RowMapper<Person> {  
  
    public Person mapRow(ResultSet rs, int rowNum)throws SQLException {  
        Person p = new Person();  
        p.setId(rs.getInt("ID"));  
        p.setFirstName(rs.getString("FNAME"));  
        p.setLastName(rs.getString("LNAME"));  
        return p;  
    }  
}
```

Les accès JDBC avec le JdbcTemplate RowMapper (3/3)

- Résultats sur **plusieurs** tuples (toujours avec le RowMapper)

```
public List<Person> fetchAllPerson() {  
    String sql = "SELECT * FROM PERSON";  
    return this.jdbcTemplate.query(sql, new PersonRowMapper());  
}
```

```
public class PersonRowMapper implements RowMapper<Person> {  
  
    public Person mapRow(ResultSet rs, int rowNum) throws SQLException {  
        Person p = new Person();  
        p.setId(rs.getInt("ID"));  
        p.setFirstName(rs.getString("FNAME"));  
        p.setLastName(rs.getString("LNAME"));  
        return p;  
    }  
}
```

Les accès JDBC avec le JdbcTemplate

RowCallbackHandler

- Spring fournit l'interface « RowCallbackHandler » dans le cas où aucun objet ne doit être retourné
 - Stocker les données dans un fichier.
 - Convertir les tuples sous format XML.
 - Appliquer des règles de filtrage sur les données

```
public interface RowCallbackHandler {  
    void processRow(ResultSet rs) throws SQLException;  
}  
  
public void processPersonAsXML() {  
    String sql = "SELECT * FROM PERSON";  
    this.jdbcTemplate.query(sql, new PersonXmlExporter());  
}  
  
public class PersonXmlExporter implements RowCallbackHandler {  
    public void processRow(ResultSet rs) throws SQLException {  
        // Parcours le tuple pour l'export en XML  
    }  
}
```

Les accès JDBC avec le JdbcTemplate ResultSetExtractor

- Spring fournit l'interface « `ResultSetExtractor` » pour traiter un ensemble de tuples
 - La classe d'implémentation est chargée d'itérer sur les tuples.

```
public interface ResultSetExtractor<T> {  
    T extractData(ResultSet rs) throws SQLException, DataAccessException;  
}
```

```
public AddressBook generateAdressBook() {  
    String sql = "SELECT * FROM PERSON";  
    return this.jdbcTemplate.query(sql, new AddressBookExtrator());  
}
```

```
public class AddressBookExtrator implements ResultSetExtractor<AddressBook> {  
    public AddressBook extractData(ResultSet rs) throws SQLException, DataAccessException {  
        AddressBook addressBook = new AddressBook();  
        while(rs.next()) {  
            // Parser les tuples  
        }  
        return addressBook;  
    }  
}
```

Les accès JDBC avec le JdbcTemplate

Conclusion

- L'interface « **RowMapper** »
 - Adapté lorsque chaque tuple est associé à un objet.
 - **Ne doit pas** faire de rs.next
 - Retourne toujours **un seul objet**
- L'interface « **RowCallbackHandler** »
 - Adapté lorsqu'aucun résultat ne doit être retourné pour chaque tuple.
 - **Doit** faire le rs.next
 - Ne retourne **rien**, fait un traitement
- L'interface « **ResultSetExtractor** »
 - Adapté lorsqu'un ensemble de tuples correspond à un seul objet.
 - **Doit** faire le rs.next
 - Retourne toujours **un seul objet**

Les accès JDBC avec le JdbcTemplate

Les inserts et updates

➤ Insertion d'un nouveau tuple

```
public void insert(Person p) {  
    String sql = "INSERT INTO PERSON VALUES (?, ?, ?)";  
    this.jdbcTemplate.update(sql, p.getId(), p.getFirstName(), p.getLastName());  
}
```

➤ Mise à jour (update) d'un tuple existant

```
public void update(Person p) {  
    String sql = "UPDATE PERSON SET LNAME = ? WHERE ID = ? ";  
    this.jdbcTemplate.update(sql, p.getLastName(), p.getId());  
}
```

➤ Méthode update sert pour

- insert, delete, update

Les accès JDBC avec le JdbcClient

- Depuis Spring 6.1, vous pouvez supplanter le **JdbcTemplate** par un **JdbcClient**.
- C'est une sorte de fusion entre le
 - Le JdbcTemplate
 - Le NamedParameterJdbcTemplate
- Il a les mêmes avantages que le JdbcTemplate mais en plus
 - Vous pouvez l'utiliser comme un Stream
 - Vous pouvez vous passer des RowMapper

Les accès JDBC avec le JdbcClient

➤ Exemple d'utilisation

@Repository

```
public class SpringJdbcPersonDaoImpl implements IPersonDao {
```

@Autowired

```
private JdbcClient jdbcClient;
```

@Override

```
public int fetchPersonCount() {  
    String sql = "SELECT COUNT(1) FROM PERSON";  
    return this.jdbcClient.sql(sql).query(Integer.class).single();  
}  
}
```


Les accès JDBC avec le JdbcClient Sans l'usage du RowMapper

➤ Résultat sur un seul tuple

```
public Person fetchPersonById(Integer pkId) {  
    String sql = "SELECT * FROM PERSON WHERE ID = ?";  
    return this.jdbcClient.sql(request).param(pkId).query(Person.class).single();  
}
```

➤ Possibilité d'utiliser les requêtes variabilisées

```
public Person fetchPersonById(Integer pkId) {  
    String sql = "SELECT * FROM PERSON WHERE ID = :id";  
    return this.jdbcClient.sql(request).param("id", pkId).query(Person.class).single();  
}
```

Les accès JDBC avec le JdbcClient Sans l'usage du RowMapper

- Résultats sur plusieurs tuples (toujours SANS le RowMapper)

```
public List<Person> fetchAllPerson() {  
    String sql = "SELECT * FROM PERSON";  
    result = this.jdbcClient.sql(sql).query(Person.class).list();  
}
```

- Il n'y a aucune annotation / marque spécifique dans la classe de mapping (*Person* ici)
- Il est tout de même important de s'assurer du bon nom des attributs de cette classe
 - Nom de l'attribut = nom de la colonne
 - ❑ firstName <=> first_name

Les accès JDBC avec le JdbcClient

Les inserts et updates

➤ Insertion d'un nouveau tuple

```
public void insert(Person p) {  
    String sql = "INSERT INTO PERSON VALUES (?, ?, ?)";  
    this.jdbcClient.sql(sql).params(p.getId(), p.getFirstName(), p.getLastName()).update();  
}
```

➤ Mise à jour (update) d'un tuple existant

```
public void update(Person p) {  
    String sql = "UPDATE PERSON SET LNAME = ? WHERE ID = ? ";  
    this.jdbcClient.sql(sql).params(p.getLastName(), p.getId()).update();  
}
```

➤ Méthode update sert pour

- insert, delete, update



Travaux pratiques

36

Réaliser les travaux pratiques suivants:

TP 10 : Faire usage du JdbcTemplate (ou JdbcClient si Spring Core $\geq$ 6.1)

Gestion des transactions avec Spring

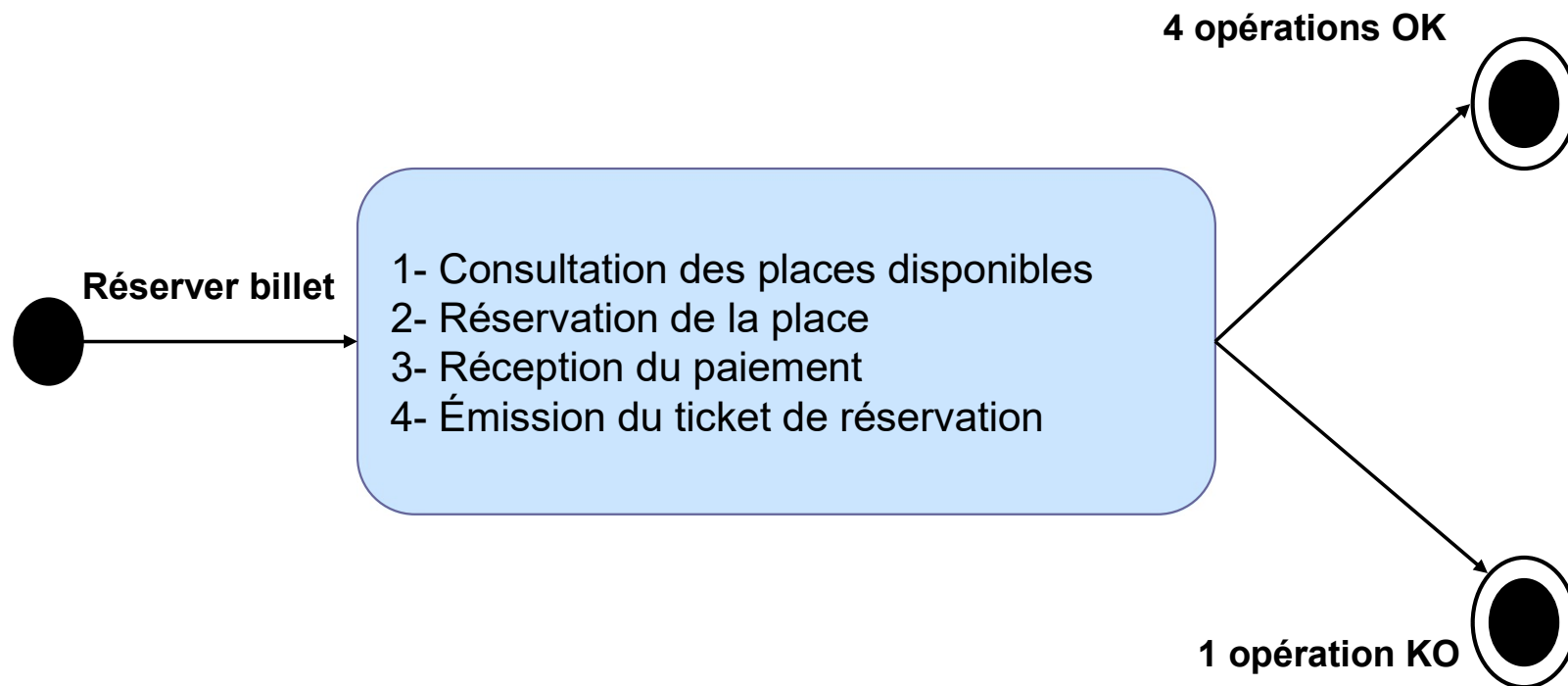
- ✓ Rappel sur la notion de transaction
- ✓ Transactionnel avec Spring
 - ✓ Via le code
 - ✓ Via les aspects
 - ✓ Via les annotations
- ✓ Composantes
 - ✓ Niveau de propagation
 - ✓ Niveau d'isolation
 - ✓ Timeout
 - ✓ Gestion des rollback

Notion de transaction

- Une transaction permet d'associer un ensemble d'opérations dans une session de travail.
 - Si les exécutions de toutes les opérations se terminent correctement, la transaction aboutit
 - Si une des opérations ne se termine pas correctement, la transaction échoue

Un exemple

➤ La réservation d'un billet de spectacle



Propriétés ACID (1/2)

➤ Propriétés fondamentales d'une transaction

➤ Atomic (Atomicité)

- Les opérations qui constituent une transaction forment une unité indivisible : soit l'ensemble des opérations est mené à terme, soit aucune opération n'est effectuée.

➤ Consistent (Cohérence)

- La base passe d'un état initial cohérent à un état final cohérent, dans le respect des règles d'intégrité référentielle définies sur la base de données.

Propriétés ACID (2/2)

➤ Isolated (Isolation)

- Les mises à jour effectuées au cours de l'exécution d'une transaction restent invisibles aux autres transactions.

➤ Durable (Durabilité)

- Le résultat de la transaction est persisté dans le système, même après l'arrêt de celui-ci.

Spring et les transactions

- Spring permet la gestion déclarative des transactions
 - Basée sur la programmation par aspects
 - Une transaction peut être vue comme un aspect qui intercepte une méthode (Around Advice)
- Spring s'intègre à de nombreux gestionnaires de transactions
 - Basés sur Hibernate, JDBC...
- Spring et JEE
 - Spring propose un gestionnaire de transactions JTA (Java Transaction API)

Transaction managers

➤ DataSourceTransactionManager

- Pour les transactions sur du JDBC
- Se lie à la DataSource

➤ HibernateTransactionManager

- Pour les transactions avec une persistance gérée par Hibernate
- Se lie à la SessionFactory
- Attention : plusieurs versions en fonction de celle d'Hibernate (3, 4, 5 ⇔
`org.springframework.orm.hibernate3,`
`org.springframework.orm.hibernate4,`
`org.springframework.orm.hibernate5`)

➤ JtaTransactionManager

- Pour les transactions sur plusieurs ressources, gérées par JTA

Transaction programmatique (1/2)

- Système de template et Callback = TransactionTemplate

```
@Service
public class MonService {
    @Autowired
    private TransactionTemplate transactionTemplate;

    public void transfertMoney() {
        transactionTemplate.execute(new TransactionCallback<Object>() {
            public Object doInTransaction(TransactionStatus ts) {
                try {
                    // Faire qqc(select, ...) ← Code transactionnel
                } catch (Exception e) {
                    ts.setRollbackOnly(); ← Déclencher RollBack
                }
                return null; ← Si succès Commit
            }
        });
    }
    ...
}
```

- Le TransactionTemplate permet de garder le code métier indépendant de l'implémentation de l'accès aux données (jdbc, hibernate, etc...)

Transaction programmatique (2/3)

- Système de template et Callback = TransactionTemplate

```
@Service
public class MonService {
    @Autowired
    private TransactionTemplate transactionTemplate;

    public void transfertMoney() {
        transactionTemplate.execute(new TransactionCallbackWithoutResult() {
            public void doInTransactionWithoutResult (TransactionStatus ts) {
                try {
                    // Faire qqc(update, delete, .. ← Code transactionnel
                } catch (Exception e) {
                    ts.setRollbackOnly(); ← Déclencher RollBack
                }
            }
        });
    }
    ...
}
```

Transaction programmatique (3/3)



➤ Déclaration dans le contexte Spring

```
<bean id="transactionManager"  
  class="org.springframework.jdbc.datasource.DataSourceTransactionManager">  
  <property name="dataSource" ref="dataSource" />  
</bean>  
  
<bean id="transactionTemplate"  
  class="org.springframework.transaction.support.TransactionTemplate">  
  <property name="transactionManager" ref="transactionManager" />  
</bean>  
  
<bean id="accountService" class="com.service.AccountServiceImpl">  
  <property name="transactionTemplate" ref="transactionTemplate" />  
</bean>
```

Transactions déclaratives

Via AOP par XML



```
<beans ...>
  <bean id="transactionManager"
    class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
    <property name="dataSource" ref="dataSource" />
  </bean>

  <tx:advice id="serviceTxAdvice" transaction-manager="transactionManager">
    <tx:attributes>
      <tx:method name="create*" propagation="REQUIRED"/>
    </tx:attributes>
  </tx:advice>

  <aop:config>
    <aop:pointcut id="serviceTransactionPointcut"
      expression="execution(* com.service.*Service.*(..))" />
    <aop:advisor advice-ref="serviceTxAdvice"
      pointcut-ref="serviceTransactionPointcut" />
  </aop:config>
</beans>
```

← Attributs
de la transaction

Les attributs transactionnels

- Ils définissent la politique et les comportements transactionnels des méthodes
 - sur la propagation des transactions
 - sur les niveaux d'isolation
 - sur les optimisations Read-only
 - sur les Timeouts

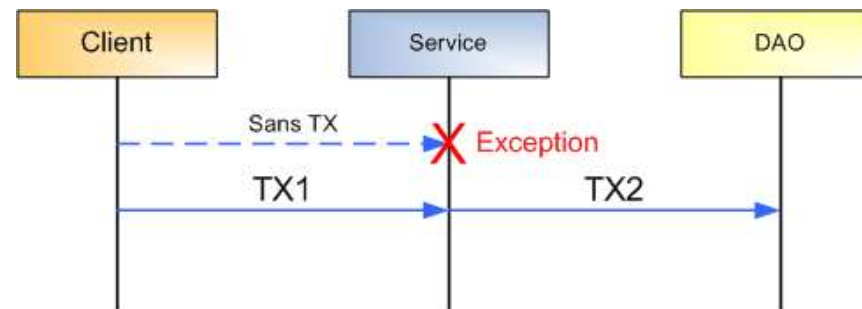
- Spring définit **7 attributs** de propagation pour les transactions
 - Ces modes ne sont pas supportés par toutes les bases de données

Configuration de la propagation

Les attributs de propagation (1/7)

➤ PROPAGATION_MANDATORY

- Indique que la méthode **doit s'exécuter** dans une transaction.
- Une exception est lancée si aucune transaction n'est définie



```
@Transactional(propagation=Propagation.MANDATORY)
```

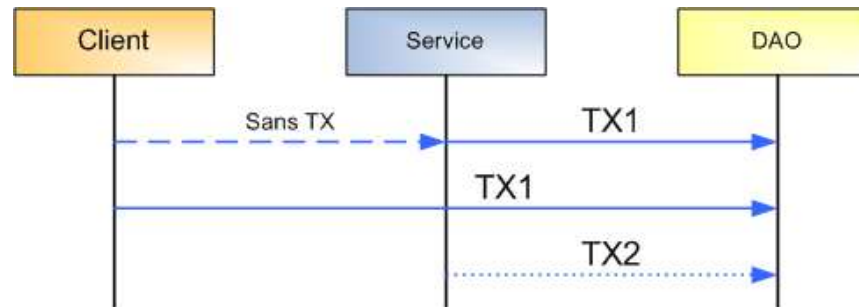
```
<tx:method name="faire*" propagation="MANDATORY"/>
```

Configuration de la propagation

Les attributs de propagation (2/7)

➤ PROPAGATION_NESTED

- Indique que la **méthode doit s'exécuter dans une transaction imbriquée** si une transaction existante est en cours. La transaction imbriquée peut avoir un Commit ou un RollBack indépendant de la transaction existante. S'il n'existe aucune transaction, se comporte comme PROPAGATION_REQUIRED



```
@Transactional(propagation=Propagation.NESTED)
```

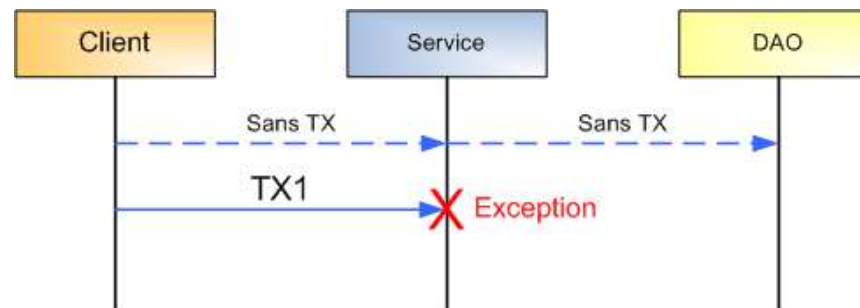
```
<tx:method name="faire*" propagation="NESTED"/>
```

Configuration de la propagation

Les attributs de propagation (3/7)

➤ PROPAGATION_NEVER

- Indique que la méthode **ne doit pas s'exécuter** dans un contexte transactionnel, **une exception est lancée** si une transaction est en cours



```
@Transactional(propagation=Propagation.NEVER)
```

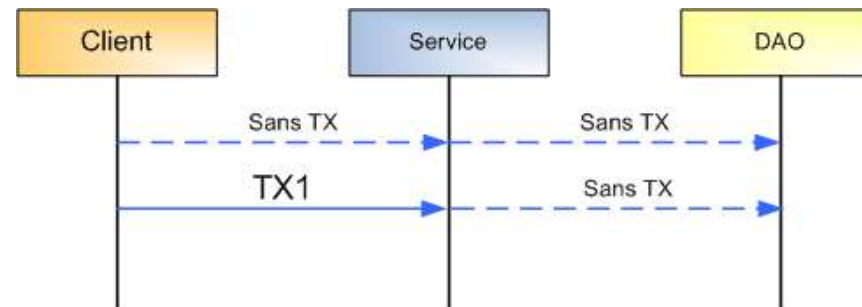
```
<tx:method name="faire*" propagation="NEVER"/>
```

Configuration de la propagation

Les attributs de propagation (4/7)

➤ PROPAGATION_NOT_SUPPORTED

- Indique que la méthode **ne doit pas s'exécuter** dans un contexte transactionnel. Si une transaction est en cours, **celle-ci est suspendue** pour la durée d'exécution de la méthode



```
@Transactional(propagation=Propagation.NOT_SUPPORTED)
```

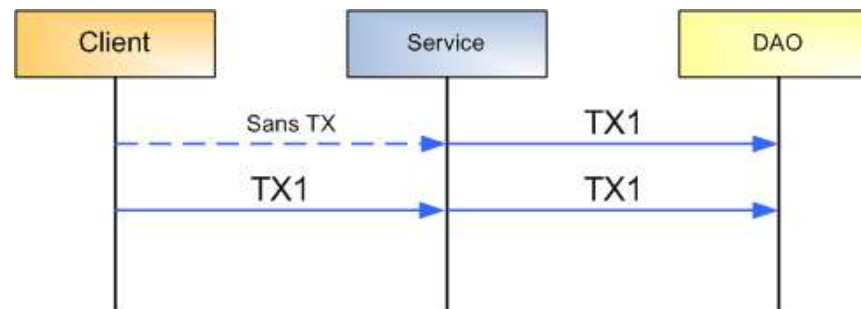
```
<tx:method name="faire*" propagation="NOT_SUPPORTED"/>
```

Configuration de la propagation

Les attributs de propagation (5/7)

➤ PROPAGATION_REQUIRED

- Indique que la méthode **doit s'exécuter** dans un contexte transactionnel. **Si une transaction est en cours, celle-ci est utilisée, sinon une nouvelle transaction est initiée.**
- Propagation par défaut.



```
@Transactional(propagation=Propagation.REQUIRED)
```

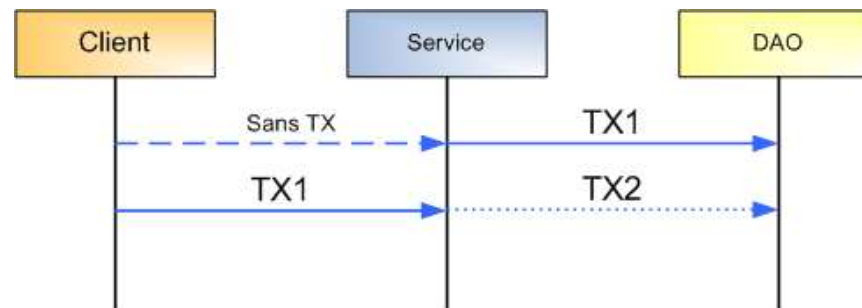
```
<tx:method name="faire*" propagation="REQUIRED"/>
```

Configuration de la propagation

Les attributs de propagation (6/7)

➤ PROPAGATION_REQUIRES_NEW

- La méthode **doit s'exécuter** dans son propre contexte transactionnel.
- Si une transaction est en cours, **une nouvelle transaction** est initiée et la transaction **existante est suspendue** pour la durée d'exécution de la méthode



```
@Transactional(propagation=Propagation.REQUIRES_NEW)
```

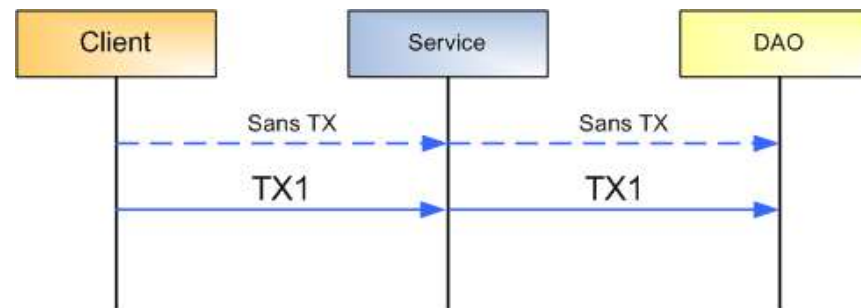
```
<tx:method name="faire*" propagation="REQUIRES_NEW"/>
```

Configuration de la propagation

Les attributs de propagation (7/7)

➤ PROPAGATION_SUPPORTS

- Indique que la méthode **n'exige pas un contexte** transactionnel, mais peut s'exécuter dans une transaction si elle existe



```
@Transactional(propagation=Propagation.SUPPORTS)
```

```
<tx:method name="faire*" propagation="SUPPORTS"/>
```

L'isolation des transactions (1/2)

- Dans une application d'entreprise, les transactions s'exécutent de manière concurrente sur des données communes
- Spring propose 4 niveaux d'isolation :
 - DEFAULT
 - READ_UNCOMMITTED
 - READ_COMMITTED
 - REPEATABLE_READ
 - SERIALIZABLE
- En Jdbc standard on a :

Isolation Level	Transactions	Dirty Reads	Non-Repeatable Reads	Phantom Reads
TRANSACTION_NONE	Not supported	Not applicable	Not applicable	Not applicable
TRANSACTION_READ_COMMITTED	Supported	Prevented	Allowed	Allowed
TRANSACTION_READ_UNCOMMITTED	Supported	Allowed	Allowed	Allowed
TRANSACTION_REPEATABLE_READ	Supported	Prevented	Prevented	Allowed
TRANSACTION_SERIALIZABLE	Supported	Prevented	Prevented	Prevented

L'isolation des transactions (2/2)

- Il est possible de paramétrer l'isolation pour chaque transaction
- Généralement, les bases de données ne supportent pas tous les niveaux de transactions
 - Si le niveau d'isolation requis n'est pas supporté par la base de données, utilisation du niveau d'isolation le plus proche.
- Une bonne isolation provoque de moins bonnes performances
 - En fonction des besoins, il faut arbitrer entre isolation et performances

Gestion des lectures sales

➤ DIRTY READ

- Lectures pouvant se produire sur des données non encore "comittées" par une autre transaction. En cas de Rollback les données obtenues sont invalides

➤ READ_UNCOMMITTED

- Niveau d'isolation le plus faible : la base de données autorise les lectures sales



La plupart des bases de données ne proposent pas le niveau READ_UNCOMMITTED

➤ READ_COMMITTED

- Protège contre les lectures sales

Lectures successives

➤ **NON REPEATABLE READ**

- Se dit d'une transaction qui exécute une même requête plusieurs fois et récupère des données pouvant être différentes. La différence étant due à une autre transaction concurrente mettant ces données à jour

➤ **REPEATABLE_READ**

- Protège contre les dirty reads et les nonrepeatable reads.

Lectures fantômes

➤ PHANTOM READ

- Se produit lorsqu'une transaction lit plusieurs lignes, pendant qu'une autre transaction concurrente insère des lignes. Sur une nouvelle requête des nouvelles lignes peuvent apparaître

➤ SERIALIZABLE

- Niveau d'isolation ACID complet, évite les dirty reads, les nonrepeatable reads et les phantom reads.

Recommandations

- Dans la situation idéale, les transactions concurrentes sont complètement isolées les unes des autres, pour éviter les problèmes cités précédemment
- Mais l'isolation parfaite affecte les performances
 - ▀ Lock des lignes et parfois des tables !
 - ▀ Le lock agressif oblige les transactions à s'attendre entre elles
- Dans certains cas, il peut être intéressant de relâcher les niveaux d'isolation.

Paramètre Read-only

- La transaction n'effectue que des opérations de lecture.
- L'intégrité des données doit être garantie par une transaction.
- Lors d'une transaction **Read-Only**, la ressource de données peut prendre certaines initiatives d'optimisation.
 - En contrepartie, les accès en écriture sont interdits pendant la durée de la transaction

Transaction Timeout (en secondes)

- Si une transaction devient particulièrement longue (bloquée sur un lock par exemple), il est possible de déclencher un RollBack après un certain temps
- Le **timer** du timeout est déclenché sur un début de transaction, cette fonctionnalité n'a donc de sens que pour les attributs de propagation suivants :
 - **PROPAGATION_REQUIRED**
 - **PROPAGATION_REQUIRES_NEW**
 - **PROPAGATION_NESTED**

Mise en place



- **Par défaut**, si l'on ne précise rien :
 - PROPAGATION_REQUIRED
 - ISOLATION_DEFAULT
 - read-only="false"
 - rollback-for="java.lang.RuntimeException"
- Il est possible de surcharger ces attributs.

```
<tx:advice id="daoTxAdvice" transaction-manager="txManager">
  <tx:attributes>
    <tx:method name="create*" propagation="REQUIRED" isolation="REPEATABLE_READ"
              rollback-for="java.lang.Exception" />

    <tx:method name="find*" propagation="REQUIRED" isolation="READ_COMMITTED"
              read-only="true" rollback-for="java.lang.Exception" />
  </tx:attributes>
</tx:advice>
```


Politique de rollback



➤ Par défaut, un Rollback est invoqué lors du lancement d'une Exception **Unchecked**

➤ Exception héritant de **java.lang.RuntimeException**

❑ Il n'est pas nécessaire de gérer ce type d'exception

➤ **Aucun Rollback** n'est effectué pour les exceptions de type `java.lang.Exception`

➤ Il est possible de spécifier un rollback pour certaines exceptions

➤ Attribut `rollback-for`. Exemple :

```
<tx:advice id="daoTxAdvice" transaction-manager="txManager">
  <tx:attributes>
    <tx:method name="create*" propagation="MANDATORY"
      rollback-for="com.service.ex.MonException"/>
  </tx:attributes>
</tx:advice>
```

Timeout



- Spring permet de spécifier un timeout de transaction (en **secondes**)
 - Par défaut, les transactions utilisent le timeout de la base de données
 - Exemple de paramétrage du timeout :

```
<tx:advice id="daoTxAdvice" transaction-manager="txManager">  
  <tx:attributes>  
    <tx:method name="create*" ... timeout="10" rollback-for="java.lang.Exception" />  
  </tx:attributes>  
</tx:advice>
```

Transactions et Annotations

- Il est possible d'utiliser les annotations pour le paramétrage des transactions

```
@Transactional (readOnly = true)
public class ClientServiceImpl implements ClientService{

    @Transactional(propagation=Propagation.REQUIRES_NEW)
    public Client findClientById(Long id) {
        ...
    }
}
```

- **Attention** à vos imports :
org.springframework.transaction.annotation.Transactional

Transactions et Annotations

- **ATTENTION** : pour que l'annotation **@Transactional** soit opérationnelle il faudra :
 - Si vous êtes en configuration XML, ajouter dans le fichier de configuration Spring
 - ❑ `<tx:annotation-driven transaction-manager="txManager"/>`
 - Si vous êtes en @Configuration, ajoutez l'annotation `@org.springframework.transaction.annotation.EnableTransactionManagement` sur votre classe de configuration
- Dans **tous les cas**, n'oubliez pas de déclarer un bean `TransactionManager`
 - Sauf en Spring Boot, qui se charge de le faire pour vous

Transactions et Annotations



➤ Exemple via la @Configuration

```
package com.banque.spring;

import javax.sql.DataSource;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.*;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;

@Configuration
@PropertySource("classpath:spring/database.properties")
@ComponentScan({ "com.banque" })
@EnableTransactionManagement
public class SpringConfigurationData {

    @Bean(destroyMethod = "close")
    public DataSource dataSource() { // ... }

    @Bean
    public JdbcTemplate jdbcTemplate(DataSource dataSource) { // ...}

    @Bean
    public DataSourceTransactionManager transactionManager(DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }
}
```

Transactions et Annotations

➤ Transaction sur une classe

```
@Transactional
public class PersonServiceImpl implements IPersonService {
    public void savePerson(Person p) {
        [...]
    }
}
```

➤ La transaction est appliquée sur toutes les méthodes de la classe.

➤ L'annotation peut être déclarée également sur l'interface.

➤ L'annotation peut être déclarée à la fois sur la classe et les méthodes

```
@Transactional(timeout=60)
public class PersonServiceImpl implements IPersonService {
    @Transactional(timeout=30)
    public void savePerson(Person p) {
        [...]
    }
}
```

Transactions et Annotations

- @Transactional supporte la gestion des exceptions

```
@Transactional(rollbackFor=PersonAccessException.class)
public Person getPersonByName(String name) {
    ...
}
```

- Précise les exceptions provoquant un rollback.

```
@Transactional(noRollbackFor=PersonAccessException.class)
public Person getPersonByName(String name) {
    ...
}
```

- Précise les exceptions ignorées pour le rollback.

```
@Transactional(rollbackFor=Throwable.class, noRollbackFor=PersonAccessException.class)
public Person getPersonByName(String name) {
    ...
}
```

- La règle la plus précise l'emporte – toute exception autre que « PersonAccessException » provoque un rollback

Transactions et Annotations

72

Nom de l'attribut	Rôle	Valeur par défaut
propagation	mode de propagation de la transaction	PROPAGATION_REQUIRED
isolation	niveau d'isolation de la transaction	ISOLATIONDEFAULT
readOnly	indique si la transaction est en lecture seule (false) ou lecture/écriture(true)	false
timeout		valeur par défaut de l'implémentation du système de gestion des transactions utilisé
rollbackFor	ensemble d'exceptions héritant de Throwable qui provoquent un rollback de la transaction si elles sont levées durant les traitements	java.lang.RuntimeException.class
rollbackForClassName	ensemble de noms de classes héritant de Throwable pour lesquelles un rollback est effectué si des exceptions sont levées pendant les traitements	"java.lang.RuntimeException"
noRollbackFor	ensemble d'exceptions héritant de Throwable qui ne provoquent pas un rollback de la transaction si elles sont levées durant les traitements	
noRollbackForClassName	ensemble de noms de classes héritant de Throwable pour lesquelles la levée d'une exception ne provoquera pas de rollback	



Travaux pratiques

73

Réaliser les travaux pratiques suivants:

TP 11 : Ajoutons les transactions à notre code

Intégration de Spring avec JPA et Hibernate

74

- ✓ Rappels
 - ✓ Relations entre tables
 - ✓ ORM
 - ✓ JPA
 - ✓ Entités et DAOs
- ✓ Spring et JPA
- ✓ Rappels Hibernate
- ✓ Spring et Hibernate

Rappel sur les relations en base de données

- Une base relationnelle met en évidence des relations entre tables
- Ces relations peuvent être de différentes cardinalités :
 - 1..1
 - 1..n
 - n..m
- La cardinalité est le nombre de participations d'une entité à une relation.

Rôle des relations dans un projet

- En fonction des cardinalités de chaque élément on aura des clefs étrangères et des clefs de jointure.
- Mais cette notion relationnelle **n'existe pas** dans le monde objet.
- Or nous allons mapper notre base de données vers des objets Java $\Leftrightarrow$ des entités

Rôle des relations dans un projet

- C'est à nous de choisir lors du mapping si
 - Un Objet Contact aura un attribut de type Adhérent ?
 - ☐ Ou un Objet Contact aura un attribut du même type que celui qui représente la clef primaire de l'objet Adhérent ?
 - Un Objet Adherent aura un attribut de type liste d'objets Contact ?
 - ☐ Ou un Objet Adherent aura un attribut de type List<?>, avec ? qui sera du même type que celui qui représente la clef primaire de l'objet Contact ?
 - Un Objet Contact aura un attribut de type liste d'objets Document ?
 - Un Objet Document aura un attribut de type Contact ?
- Le choix n'est pas toujours simple et peut facilement changer pendant la durée de vie du projet. Restez simple et fonctionnellement logique.

Rôle des relations dans un projet

- En plus du choix des règles de jointures, il faudra définir **un sens de lecture** à vos relations
 - Quand je mets à jour un Adherent, est-ce que cela doit automatiquement mettre à jour les éléments concernés dans la table Contact
 - Ou inversement, quand je vais mettre à jour un élément Contact, cela doit-il mettre à jour l'Adherent concerné.
- Ici aussi il n'y a pas de bon ou de mauvais choix, vous déciderez en fonction de vos besoins.

Rôle des relations dans notre projet

➤ Conseils en début de projet :

- Restez simple dans vos bases de données
- Ayez toujours une clef primaire pour chaque table
- N'abusez pas des relations N..M
- Evitez au maximum les clefs primaires sur plusieurs colonnes (hormis dans les tables de jointure)
- Gardez en tête les problématiques de volumes
- Evitez les triggers ou colonnes calculées

Rôle des relations dans notre projet

➤ Conseils en cours de projet :

- Avant de modifier ou d'introduire une nouvelle relation, regardez si une vue ne ferait pas l'affaire
 - ❑ Attention : aux problématiques de performances
- Avant d'ajouter une relation vérifiez soigneusement que cela ne va pas engendrer de cycle ($A \rightarrow B \rightarrow C \rightarrow A$)
- Avant de supprimer une relation vérifiez bien qu'elle n'est vraiment pas utile
- Ne renommez pas vos éléments à la va-vite
 - ❑ Renommer = impact immédiat dans le code

Le MAPPING Objet RELATIONNEL

➤ Object Relational Mapping

- Chaque table sera représentée par un objet
 - ❑ Sauf dans le cas des tables de jointures sans autres colonnes que les clefs primaires des tables jointurées
- Chaque attribut de l'objet représentera une colonne de la table
 - ❑ On fera toujours usage des **Wrapper** et **JAMAIS** des types primitifs
 - ❑ On choisira les contraintes relationnelles en fonction du besoin
- Les méthodes *hashCode* et *equals* devront être implémentées afin de garantir un bon comportement du framework ORM

JPA

82

- Pour **Java Persistence API**, né en 2006. La version actuelle est la 2.1 datant de 2013
- Mécanisme standard Java pour réaliser du Mapping Objet Relationnel (ORM).
- Développée dans le cadre de la version 3.0 des EJB elle peut aussi être mise en œuvre dans des applications Java.
- On ne code plus en JDBC
 - Plus de `java.sql.Connection` ou `Statement`, `ResultSet` ...
- Le mapping se réalise par le biais d'annotations (`@Entity`, `@Id`,)

JPA

83

- Les requêtes se font en JPQL, cela ressemble à du SQL, mais vous faites des requêtes sur
 - Vos objets Java (et non plus vos tables)
 - Vos attributs (et non plus vos colonnes)
 - https://docs.oracle.com/cd/E11035_01/kodo41/full/html/ejb3_lang_ref.html
- Le JPA est une norme, il faut une implémentation de cette norme pour que votre mapping fonctionne.
 - OpenJPA
 - Hibernate

Rappel JPA : Entités

- Une Entité (Entity) est
 - Une classe Java toute simple
 - C'est une classe bête, qui n'a aucune intelligence, elle ne sait rien faire
 - Elle représente une donnée en base ($\Leftrightarrow$ une ligne d'une table)
 - En général, pour une table on a une entité
 - Son rôle = représenter les données = transporter les données
- En JPA, elle sera annotée par @Entity et @Table
- Elle **NE DOIT PAS** porter d'annotations liées aux mécaniques de flux
 - @JsonXxx par exemple

Rappel JPA : Entités

- Exemple : La classe Java *UtilisateurEntity* représente un élément de la table *utilisateur*

```
1 package com.banque.entity;  
2  
3 import java.io.Serializable;  
4 import java.sql.Date;  
5 import java.sql.Timestamp;  
6 import java.util.Set;  
7  
8 import javax.persistence.CascadeType;  
9 import javax.persistence.Column;  
10 import javax.persistence.Entity;  
11 import javax.persistence.FetchType;  
12 import javax.persistence.GeneratedValue;  
13 import javax.persistence.GenerationType;  
14 import javax.persistence.Id;  
15 import javax.persistence.OneToMany;  
16 import javax.persistence.Table;  
17  
18 /**  
19  * Le bean qui represente un utilisateur. <br>  
20  */  
21 @Entity  
22 @Table(name = "utilisateur")  
23 public class UtilisateurEntity implements Serializable {  
24
```

Les imports doivent venir de
javax.persistence (JEE8-)
Ou
jakarta.persistence (JEE9+)

Ici apparait le nom de la
table (potentiellement le
schéma/catalogue)

Rappel JPA : Entités

➤ Une Entité (Entity)

- Est généralement sérialisable
 - ❑ Elle implémente `java.io.Serializable`
 - ❑ Elle a un attribut `private static final long serialVersionUID` annoté avec `@Serial`
- Possède un constructeur public sans paramètre (celui par défaut)
- Possède des `get/set` sur ses attributs
- Possède ses méthodes **`equals`** et **`hashCode`** overrideées en fonction du besoin

Rappel JPA : Les attributs d'une Entité

- Les attributs d'une Entité
 - Représentent les colonnes en base de données
 - Leur typage sera toujours de la famille des `java.lang.Object`
 - ❑ **Ne pas utiliser les types primitifs (sauf pour les clefs primaires)**
- ATTENTION : évitez des valeurs d'initialisation par défaut.
- Ils seront annotés par
 - `@Id` : pour la clef primaire
 - `@GeneratedValue` : pour définir comment la clef primaire est générée
 - `@Column` : pour le mapping avec une colonne d'une table

Rappel : Les attributs d'une Entité

```
27 @Id
28 @GeneratedValue(strategy = GenerationType.AUTO)
29 private Integer id;
30
31 @Column(name = "login", length = 120, nullable = false)
32 private String login;
33 @Column(name = "password", length = 120, nullable = false)
34 private String password;
35 @Column(name = "nom", length = 120)
36 private String nom;
37 @Column(name = "prenom", length = 120)
38 private String prenom;
39 @Column(name = "sex")
40 private Boolean sex;
41 @Column(name = "derniereConnection", length = 19)
42 private Timestamp derniereConnection;
43 @Column(name = "dateDeNaissance", length = 10)
44 private Date dateDeNaissance;
45 @Column(name = "adresse", length = 500)
46 private String adresse;
47 @Column(name = "telephone", length = 20)
48 private String telephone;
49 @Column(name = "codePostal")
50 private Integer codePostal;
51 @OneToMany(mappedBy = "utilisateur", fetch = FetchType.LAZY, cascade = CascadeType.ALL, orphanRemoval = true)
52 private Set<CompteEntity> comptes;
```

Clef primaire auto incrémentée
par la base

Relation de jointure
entre l'utilisateur et
sa liste de comptes
bancaires

Rappel JPA : Les attributs d'une Entité

- Il vous appartient d'indiquer, ou non, les contraintes d'intégrité associées aux attributs
 - Taille maximale (attribut **length** de l'annotation `@Column`)
 - Null ou pas (attribut **nullable** de l'annotation `@Column`)
 - La précision et l'échelle pour les chiffres (attribut **precision** et **scale** de l'annotation `@Column`)
 - L'unicité de la valeur (attribut **unique** de l'annotation `@Column`)
- En les indiquant :
 - Votre code sera plus robuste
 - Vous pourrez recréer un script de création de base à partir de votre code Java
 - Mais, en AUCUN CAS, elles ne serviront de mécanique de validation automatique

Rappel JPA : Les attributs d'une Entité

- Il vous appartient d'indiquer ou non les jointures
 - **@OneToOne** : pour une relation 1 - 1
 - **@OneToMany** et **@ManyToOne** pour les relations 1 - N
 - **@ManyToMany** pour les relations N - M

- En fonction des problématiques, vous complèterez via
 - l'annotation **@JoinColumn** (**@OneToOne**, **@ManyToOne**)
 - l'annotation **@JoinTable** (**@OneToMany**, **@ManyToMany**)
 - L'attribut **mappedBy** de l'annotation relationnelle (**@OneToOne**)

Rappel JPA : Les attributs d'une Entité

- Dans mon projet de vente en ligne, je peux mettre un jour un objet en vente.
 - Que se passe-t-il quand :
 - ❑ Utilisateur 1 : regarde l'objet mis en vente puis l'édite pour le changer
 - ❑ Utilisateur 2 : regarde aussi l'objet mis en vente en même temps que l'utilisateur 1 puis l'édite pour le changer après l'utilisateur 1
- C'est la problématique de *versionning* des objets
 - Avant de manipuler mon Objet mon code doit s'assurer qu'il n'a pas changé d'état en base afin de ne pas se retrouver dans un état incohérent

Rappel JPA : Les attributs d'une Entité

- En JPA vous avez deux options pour gérer cette problématique
- **@Version** : cette annotation se place sur un attribut de vos entités et va être gérée par le JPA
 - Il y aura une colonne en base de données associée à cet attribut
 - Il sera géré automatiquement par JPA lors des update/delete
 - En cas de problème un OptimisticLockException sera levé

Rappel JPA : Les attributs d'une Entité

➤ Utilisation des verrous :

- **LockModeType** : Quand vous appelez l'entity manager vous pouvez indiquer un LockModeType (LockModeType.PESSIMISTIC_WRITE ou LockModeType.OPTIMISTIC) afin de verrouiller la ligne que vous êtes en train de lire
 - ❑ LockModeType.PESSIMISTIC_WRITE : verrouillage complet
 - ❑ LockModeType.OPTIMISTIC : verrouillage en fonction de la version
- Le lock persiste tant que la transaction n'est pas terminée

Spring et les entités

- Spring ne **s'occupera en rien** de vos entités
- Leur code ne change pas parce que vous passez en Spring
- Le Spring ne s'occupe **SURTOUT** pas des liens entre entités (Relations n..m). C'est votre ORM qui s'en charge.
- Il n'y a pas obligation de faire des interfaces pour vos entités.
 - Avec JPA c'est même fortement déconseillé
 - Avec Hibernate cela dépend si vous voulez un proxy ou pas

Rappel JPA : DAO / Repository

- Pour **Data Access Object**, ancien nom pour Repository (d'où l'annotation)
- Le **SEUL** endroit où vous avez le droit de réaliser des accès à la base de données c'est dans le DAO
- Les requêtes se feront dans le DAO en
 - JPQL, si vous êtes en JPA (ou un équivalent) cela ressemble à du SQL, mais vous faites des requêtes sur vos Objets Java et non plus vos tables
 - En SQL si vous êtes en JDBC
 - En code pur, à travers des objets d'abstractions représentant la logique SQL
- JPA ou Hibernate la gestion des `java.sql.Connection` **ne sera plus** de votre ressort
- Le DAO **ne devrait pas** gérer les transactions
 - C'est l'appelant des méthodes du DAO qui a la responsabilité de gérer les transactions

Rappel JPA : DAO

- En JPA vos DAO manipulent un objet **EntityManager** fourni par un **EntityManagerFactory**
- En JPA, pour les transactions, vous faites usage d'un objet **EntityTransaction**
 - Rappels :
 - ☐ en principe pas de transactions au niveau des DAOs
 - ☐ JPA ne fait rien si vous n'êtes pas dans une transaction lors d'un ordre update/delete/insert
- La configuration des accès bases se fait dans le fichier META-INF/persistence.xml

Rappel JPA : DAO et les Requêtes

- **L'EntityManager** sait réaliser toutes les méthodes **CRUD**
- Si vous avez besoin de faire plus que du CRUD, vous pouvez écrire des requêtes en JPA
- Vous ferez usage d'un objet **Query** qui vous sera donné par **l'EntityManager**
- **Attention** : ce n'est pas du SQL, les noms représentant vos critères sont ceux de vos objets/attributs

Rappel JPA : DAO et les Requêtes

➤ Pour récupérer plusieurs objets :

➤ `getResultList`

```
Query query = em.createQuery("SELECT c FROM Country c");  
// On peut tout de même ici indiquer List<Country> si on le souhaite  
List results = query.getResultList();
```

Rappel JPA : DAO et les Requêtes dynamiques

- Dans tous les cas, vous pouvez faire usage de requêtes à trous (~PreparedStatement mais en plus efficace)

```
Query<Country> query = em.createQuery("SELECT c FROM Country c WHERE c.name = :name");  
query.setParameter("name", name);  
List result = query.getResultList();
```

- Vous ne pouvez avoir des trous que sur des VALEURS
 - ➡ Pas sur les noms d'attributs ou d'objets
- Attention : lors du setParameter on ne précise plus les ':'
- Remarque : l'objet Query<?> est pour JPA > 2, pour les versions antérieures il faut caster le type de retour en ce qui nous intéresse.

Rappel JPA : DAO et les Requêtes nommées

- Si vous le souhaitez, vous pouvez aussi faire usage de requêtes nommées
 - Aucun rapport avec des procédures stockées
- Elles se déclarent dans l'entité et s'appellent dans le DAO
- Déclaration dans l'entité via l'annotation **@NamedQuery**
- Appel dans le DAO via la méthode `createNamedQuery` de l'`EntityManager`

Rappel JPA : DAO et les Requêtes nommées

➤ Exemple de déclaration

➡ Une requête

```
@Entity
@NamedQuery(name="Country.findAll", query="SELECT c FROM Country c")
public class Country {
    ...
}
```

➡ Plusieurs requêtes

```
@Entity
@NamedQueries({
    @NamedQuery(name="Country.findAll", query="SELECT c FROM Country c"),
    @NamedQuery(name="Country.findByName", query="SELECT c FROM Country c WHERE c.name = :name"),
})
public class Country {
    ...
}
```

Rappel JPA : DAO et les Requêtes nommées

➤ Exemple d'appel

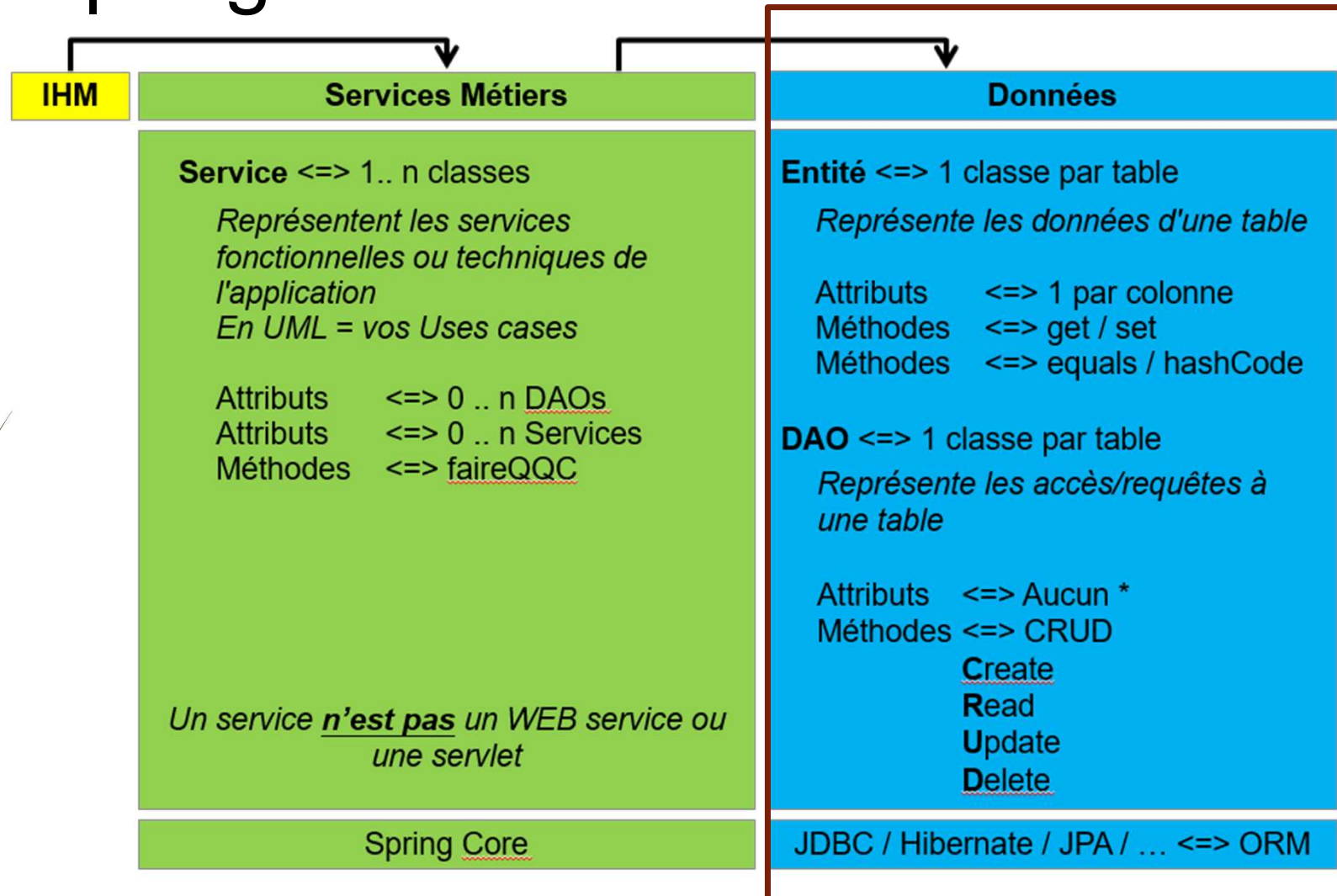
```
// Vous pouvez aussi utiliser une Query standard  
  
Query<Country> query = em.createNamedQuery("Country.findAll");  
List results = query.getResultList();
```

- Remarque : l'objet Query<?> est pour JPA > 2, pour les versions antérieures il faut caster le type de retour en ce qui nous intéresse.

Spring et les DAOs

- Un DAO / repository est un objet **géré** par le Spring
- Chaque DAO doit être représenté par une interface
 - Sinon vous ne faites pas de l'IoC
 - Sinon vous ne pourrez pas bénéficier des avantages de l'AOP
- Il devra donc soit (l'un **ou** l'autre) :
 - être déclaré comme bean (en <bean> ou @Bean)
 - être annoté par @Repository (solution à privilégier)
- Dans tous les cas, vous ne devrez **JAMAIS** instancier un DAO, c'est au Spring de le faire.
 - Via une injection de dépendance manuelle (gérée par vous en XML ou @Configuration)
 - Via l'annotation @Autowired dans vos classes de services métiers (solution à privilégier)

Spring et les DAOs



Spring et JPA

- La configuration JPA sera laissée au Spring, il s'occupera de EntityManagerFactory et de l'EntityManager
- Il est très simple de faire passer un projet JPA pur vers un projet Spring-JPA
 - Tout se passe dans les fichiers de configurations du Spring
- On y gagnera surtout sur la gestion des transactions et des tests unitaires

Configuration JPA avec Spring En XML



- Configuration de l' « entityManagerFactory » au sein du contexte Spring

```
...
<bean id="jpaPersonDaoImpl" class="exemple.JpaPersonDaoImpl">
  <property name="entityManagerFactory" ref="entityManagerFactory" />
</bean>

<bean id="entityManagerFactory" class="org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean">
  <property name="dataSource" ref="dataSource" />
  <property name="packagesToScan" value="com.banque.entity.impl" />
  <property name="persistenceProviderClass" value="org.hibernate.jpa.HibernatePersistenceProvider" />
  <property name="jpaProperties">
    <props>
      <prop key="hibernate.dialect.storage_engine">${hibernate.dialect.storage_engine}</prop>
      <prop key="hibernate.connection.handling_mode">${hibernate.connection.handling_mode}</prop>
      <prop key="hibernate.connection.pool_size">${hibernate.connection.pool_size}</prop>
      <prop key="hibernate.cache.provider_class">${hibernate.cache.provider_class}</prop>
      <prop key="hibernate.cache.use_second_level_cache">${hibernate.cache.use_second_level_cache}</prop>
      <prop key="hibernate.show_sql">false</prop>
    </props>
  </property>
</bean>

<bean id="dataSource" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
  <property name="driverClassName" value="${...}" />
  <property name="url" value="${...}" />
  ...
</bean>
```

Configuration JPA avec Spring

En @Configuration



```
@Configuration
@PropertySource("classpath:spring/database.properties")
@ComponentScan({ "com.banque.dao", "com.banque.service" })
@EnableTransactionManagement
public class SpringConfigurationData {
```

```
@Autowired
private Environment env;
```

```
@Bean(destroyMethod = "close")
public DataSource dataSource() {
    BasicDataSource resu = new BasicDataSource();
    resu.setDriverClassName(this.env.getProperty("db.driver"));
    resu.setUrl(this.env.getProperty("db.url"));
    resu.setUsername(this.env.getProperty("db.login"));
    resu.setPassword(this.env.getProperty("db.password"));
    return resu;
}
```

```
...
```

```
@Bean
public LocalContainerEntityManagerFactoryBean entityManagerFactory() {
    LocalContainerEntityManagerFactoryBean em = new LocalContainerEntityManagerFactoryBean();
    em.setDataSource(this.dataSource());
    em.setPackagesToScan("com.banque.entity.impl");
    em.setPersistenceProviderClass(org.hibernate.jpa.HibernatePersistenceProvider.class);
    Properties jpaProperties = new Properties();
    jpaProperties.put("hibernate.dialect.storage_engine", this.env.getProperty("hibernate.dialect.storage_engine"));
    jpaProperties.put("hibernate.connection.handling_mode",
        this.env.getProperty("hibernate.connection.handling_mode"));
    jpaProperties.put("hibernate.cache.provider_class", this.env.getProperty("hibernate.cache.provider_class"));
    jpaProperties.put("hibernate.cache.use_second_level_cache",
        this.env.getProperty("hibernate.cache.use_second_level_cache"));
    jpaProperties.put("hibernate.hibernate.show_sql", Boolean.FALSE);
    jpaProperties.put("hibernate.connection.pool_size",
        Integer.valueOf(this.env.getProperty("hibernate.connection.pool_size", "5")));
    em.setJpaProperties(jpaProperties);
    return em;
}
```

```
@Bean
public PlatformTransactionManager transactionManager(EntityManagerFactory entityManagerFactory) {
    // EntityManagerFactory vient de la factory LocalContainerEntityManagerFactoryBean, qui est un FactoryBean.
    return new org.springframework.orm.jpa.JpaTransactionManager(entityManagerFactory);
}
```

Configuration JPA avec Spring

➤ Notez que

- la déclaration de la **dataSource** est la même qu'en JDBC
- Nous faisons ici usage d'un objet factory ⇔ ref fera le lien vers un objet créé par la factory et pas la factory elle-même
⇔ Ici un `javax.persistence.EntityManagerFactory`

- ## ➤ Toutes les informations qui étaient dans le fichier `persistence.xml` sont passées dans la configuration
- On peut l'effacer du projet

Remarque : Les FactoryBean en Spring

- *LocalContainerEntityManagerFactoryBean* est un objet qui implémente l'interface `org.springframework.beans.factory.FactoryBean<EntityManagerFactory>`
- Comme tous les objets de type `FactoryBean`, l'injection peut se faire sur :
 - L'objet lui-même (rien de spécial)
 - Ou, l'objet créé par la factory (c'est ce qui se passe ici lors de la déclaration du `transactionManager`)

Configuration JPA avec Spring

➤ Le DAO JPA peut posséder

➤ une méthode *set* ou un attribut de type `javax/jakarta.persistence.EntityManagerFactory`



➤ Ou, directement un attribut `EntityManager`

❑ Dans ce cas, le lien pourra être géré directement par Spring via l'annotation JPA `@javax/jakarta.persistence.PersistenceContext`



Configuration JPA avec Spring

➤ Côté transactionnel

- Vous pouvez garder vos EntityTransaction si vous le souhaitez
- Vous pouvez faire usage de l'annotation `@Transactional`
 - ❑ N'oubliez pas le `<tx:annotation-driven transaction-manager="transactionManager" />`
- Vous pouvez faire usage des aspects

➤ Dans les deux derniers cas, vous aurez besoin d'un transaction manager JPA qui sera lié à votre EntityManagerFactory

```
<bean id="transactionManager" class="org.springframework.orm.jpa.JpaTransactionManager">
  <property name="entityManagerFactory" ref="entityManagerFactory" />
</bean>
```

```
@Bean
public PlatformTransactionManager transactionManager(EntityManagerFactory entityManagerFactory) {
  return new org.springframework.orm.jpa.JpaTransactionManager(entityManagerFactory);
}
```

Hibernate

- Hibernate est une solution open source de type ORM (**O**bject **R**elational **M**apping)
 - Géré par Red Hat (comme JBoss)
 - Né en 2003, actuellement en version 6 (version de 2022)
- Hibernate sait représenter une base de données en objets java et inversement
- Hibernate est
 - Un framework ORM avec ses propres classes
 - Une implémentation de JPA
- Hibernate est compatible avec les standards JSE et JEE
 - Compatibilité avec tous les serveurs d'application JEE du marché
 - ❑ Websphere, Weblogic, JBoss, etc..
 - S'intègre avec JNDI, JDBC, JTA

Hibernate / JPA

➤ Hibernate peut fonctionner de plusieurs manières :

➤ Hibernate pur : tous les imports sont ceux d'Hibernate

- ☐ Entités décrites en XML propriétaire Hibernate
- ☐ DAOs faisant usage des objets d'Hibernate

➤ JPA pur : tous les imports sont ceux de JPA

- ☐ Possible uniquement à partir de la version 5.1 d'Hibernate
- ☐ Entités décrites en annotation JPA
- ☐ DAOs faisant usage des objets JPA

➤ Hybride : des imports JPA, des imports Hibernate

- ☐ Entités décrites en annotation JPA et quelques annotations Hibernate
- ☐ DAOs faisant usage des objets JPA et Hibernate

Ancien projet

Projet Idéal

Projet Réel

Hibernate

- Hibernate supporte les annotations JPA 2.0
 - Depuis la version 3.5 d'Hibernate.
 - Package « javax.persistence.* » / « jakarta.persistence.* »
 - Couvre un large spectre des besoins en matière de persistance.

- Hibernate fournit des extensions spécifiques
 - S'ajoutent aux annotations JPA.
 - ❑ Gestion des performances spécifiques Hibernate.
 - ❑ Fonctionnalités non implémentées par JPA.
 - Moins indispensables depuis JPA 2.0

Rappel : Hibernate

➤ Session

- Gère la persistance des objets au sein d'une unité de travail.
- Object de type « stateful ».

➤ SessionFactory

- Sert de fabrique pour les Sessions.
- Encapsule les mappings sur les objets et la base de données.
- « Thread safe » et peut donc être partagé.
- Bean de type « Singleton » sous Spring.

Rappel Hibernate : Entités

- La notion d'entité est la même en Hibernate et JPA
- Vous avez le choix :
 - Faire vos entités comme en JPA
 - ❑ Possible à 100% depuis hibernate 5
 - Faire vos entités via un fichier descriptif XML propre à hibernate
 - Faire vos entités comme en JPA + quelques annotations Hibernate
 - ❑ Pratique si vous êtes en JPA 1.0 ou 2.0

Rappel Hibernate : Entités

- En hibernate vos entités peuvent être représentées par des interfaces
 - Ce n'est pas une obligation
 - De cette manière vous pouvez utiliser les caches et proxy d'Hibernate
- Comme en JPA Spring **ne s'occupe pas** de vos entités, c'est votre problème pas le sien

Rappel Hibernate : Entités

➤ Exemple de mapping en XML

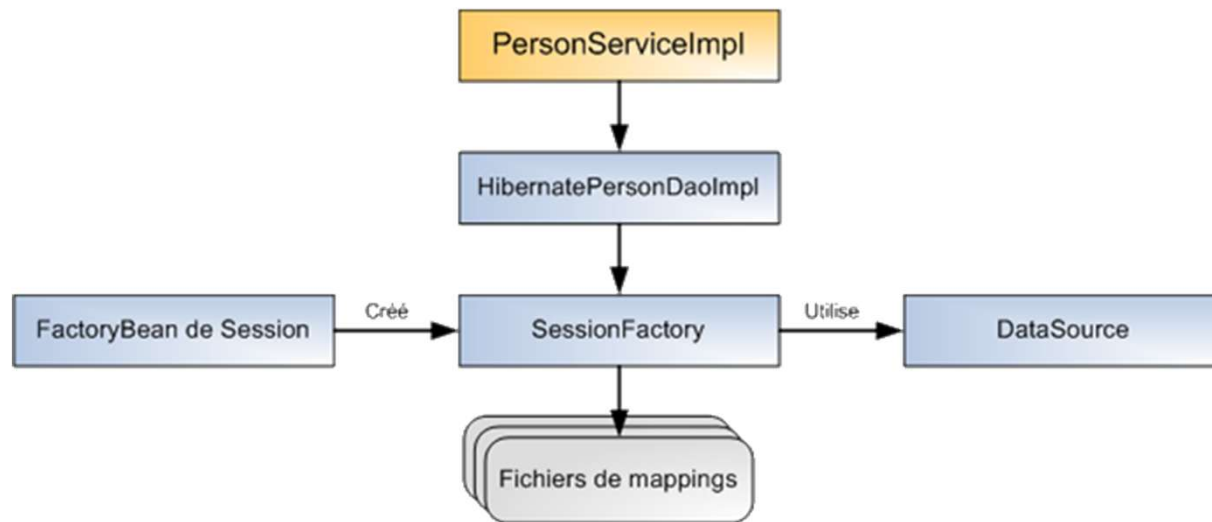
```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN" "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
3 <hibernate-mapping>
4   <class name="com.banque.entity.impl.UtilisateurEntity" table="utilisateur">
5     <id name="id" type="java.lang.Integer">
6       <column name="id" />
7       <generator class="native" />
8     </id>
9     <property name="login" type="string">
10      <column name="login" length="120" not-null="true" />
11    </property>
12    <property name="password" type="string">
13      <column name="password" length="120" not-null="true" />
14    </property>
15    <property name="nom" type="string">
16      <column name="nom" length="120" />
17    </property>
18    <property name="prenom" type="string">
19      <column name="prenom" length="120" />
20    </property>
21    <property name="sex" type="java.lang.Boolean">
22      <column name="sex" />
23    </property>
```

Rappel Hibernate : DAO

- En Hibernate vos DAO manipulent un objet **Session** fourni par un **SessionFactory**
- En Hibernate, pour les transactions, vous faites usage d'un objet `SharedSessionContract`
 - Rappels :
 - ❑ en principe pas de transactions aux niveaux des DAOs
- La configuration des accès bases se fait dans le fichier `hibernate.cfg.xml` (ou un fichier `properties`)

Rappel Hibernate : DAO

- La persistance avec Hibernate requiert l'accès à une **SessionFactory**
 - Une datasource locale ou gérée par un moteur transactionnel.
- Spring met à disposition un bean pour la configuration de la « SessionFactory »



Configuration d'Hibernate avec Spring En XML



➤ Configuration de la « sessionFactory » au sein du contexte Spring

```
<bean id="HibernatePersonDaoImpl" class="exemple.HibernatePersonDaoImpl">
  <property name="sessionFactory" ref="sessionFactory" />
</bean>

<bean id="sessionFactory" class="org.springframework.orm.hibernate5.LocalSessionFactoryBean">
  <property name="dataSource" ref="dataSource" />
  <property name="annotatedClasses">
    <list>
      <value>exemple.Person</value>
      <value>exemple.AddressBook</value>
    </list>
  </property>
  <property name="hibernateProperties">
    <props>
      <prop key="hibernate.dialect.storage_engine">${hibernate.dialect.storage_engine}</prop>
      <prop key="hibernate.connection.handling_mode">${hibernate.connection.handling_mode}</prop>
      <prop key="hibernate.connection.pool_size">${hibernate.connection.pool_size}</prop>
      <prop key="hibernate.cache.provider_class">${hibernate.cache.provider_class}</prop>
      <prop key="hibernate.cache.use_second_level_cache">${hibernate.cache.use_second_level_cache}</prop>
      <prop key="hibernate.show_sql">false</prop>
    </props>
  </property>
</bean>

<bean id="dataSource" class="..."> ... </bean>
```

- L'attribut « annotatedClasses » spécifie de façon explicite les classes persistantes (annotations JPA et/ou Hibernate).

Configuration d'Hibernate avec Spring En XML



➤ Configuration de la « sessionFactory » avec l'attribut « packagesToScan »

```
<bean id="sessionFactory" class="org.springframework.orm.hibernate5.LocalSessionFactoryBean">
  <property name="dataSource" ref="datasource" />
  <property name="packagesToScan" value="com.app.entity"/>

  <property name="hibernateProperties">
    <props>
      <prop key="...">...</prop>
    </props>
  </property>

</bean>
```

Configuration d'Hibernate avec Spring

En @Configuration



```
@Configuration
@PropertySource("classpath:spring/database.properties")
@ComponentScan({ "com.banque.dao", "com.banque.service" })
@EnableTransactionManagement
public class SpringConfigurationData {

    @Bean(destroyMethod = "close")
    public DataSource dataSource(@Value("${db.driver}") String driver, @Value("${db.url}") String url, @Value("${db.login}") String login, @Value("${db.password}") String pwd) {
        BasicDataSource resu = new org.apache.commons.dbcp2.BasicDataSource();
        resu.setDriverClassName(driver); resu.setUrl(url); resu.setUsername(login); resu.setPassword(pwd);
        return resu;
    }

    @Bean
    public org.springframework.orm.hibernate5.LocalSessionFactoryBean sessionFactory(DataSource dataSource, @Value("${hibernate.dialect}") String dialect,
        @Value("${hibernate.connection.pool_size}") Integer pSize, @Value("${hibernate.cache.use_second_level_cache}") Boolean useCache) {
        org.springframework.orm.hibernate5.LocalSessionFactoryBean resu = new org.springframework.orm.hibernate5.LocalSessionFactoryBean();
        resu.setDataSource(dataSource);
        resu.setMappingResources("hibernate/compte.hbm.xml", "hibernate/operation.hbm.xml", "hibernate/utilisateur.hbm.xml");
        Properties hibernateProperties = new Properties();
        hibernateProperties.put("hibernate.dialect", dialect);
        hibernateProperties.put("hibernate.connection.pool_size", pSize);
        hibernateProperties.put("hibernate.cache.use_second_level_cache", useCache);
        resu.setHibernateProperties(hibernateProperties);
        return resu;
    }

    @Bean
    public org.springframework.orm.hibernate5.HibernateTransactionManager transactionManager(SessionFactory sessionFactory) {
        return new org.springframework.orm.hibernate5.HibernateTransactionManager(sessionFactory);
    }
}
```

Configuration d'Hibernate avec Spring

- L'implémentation dans Spring dépendra de la version Hibernate
 - `org.springframework.orm.hibernate3`
 - `org.springframework.orm.hibernate4`
 - `org.springframework.orm.hibernate5`
- Impact sur
 - L'objet sessionFactory
 - ❑ `org.springframework.orm.hibernate4.LocalSessionFactoryBean`
 - **ET** l'objet transactionManager
 - ❑ `org.springframework.orm.hibernate4.HibernateTransactionManager`

Configuration d'Hibernate avec Spring

➤ ATTENTION

➤ Si vous êtes en **Spring Boot**, n'oubliez pas que **c'est lui qui gère la version d'Hibernate**

❑ En Spring Boot 2.2.2 vous êtes en Hibernate 5.4.9

❑ En Spring Boot 3.0.0 vous êtes en Hibernate 6.1.5

➤ Tout **changement** de version d'Hibernate aura un impact **très FORT** sur votre couche DAO/Entité

➤ Faites des tests sur ces couches pour simplifier vos migrations

Configuration d'Hibernate avec Spring

➤ DAO

@Repository

```
public class SpringHibernatePersonDaoImpl implements IPersonDao {
```

@Autowired

```
private SessionFactory sessionFactory;
```

```
protected final Session getCurrentSession() {  
    return sessionFactory.getCurrentSession();  
}
```

```
public Person fetchPersonByLastName(String lname) {  
    Query query = this.getCurrentSession().createQuery("from person where lastName =  
?");  
    query.setString("lastName", lname);  
    return (Person) query.uniqueResult();  
}  
}
```

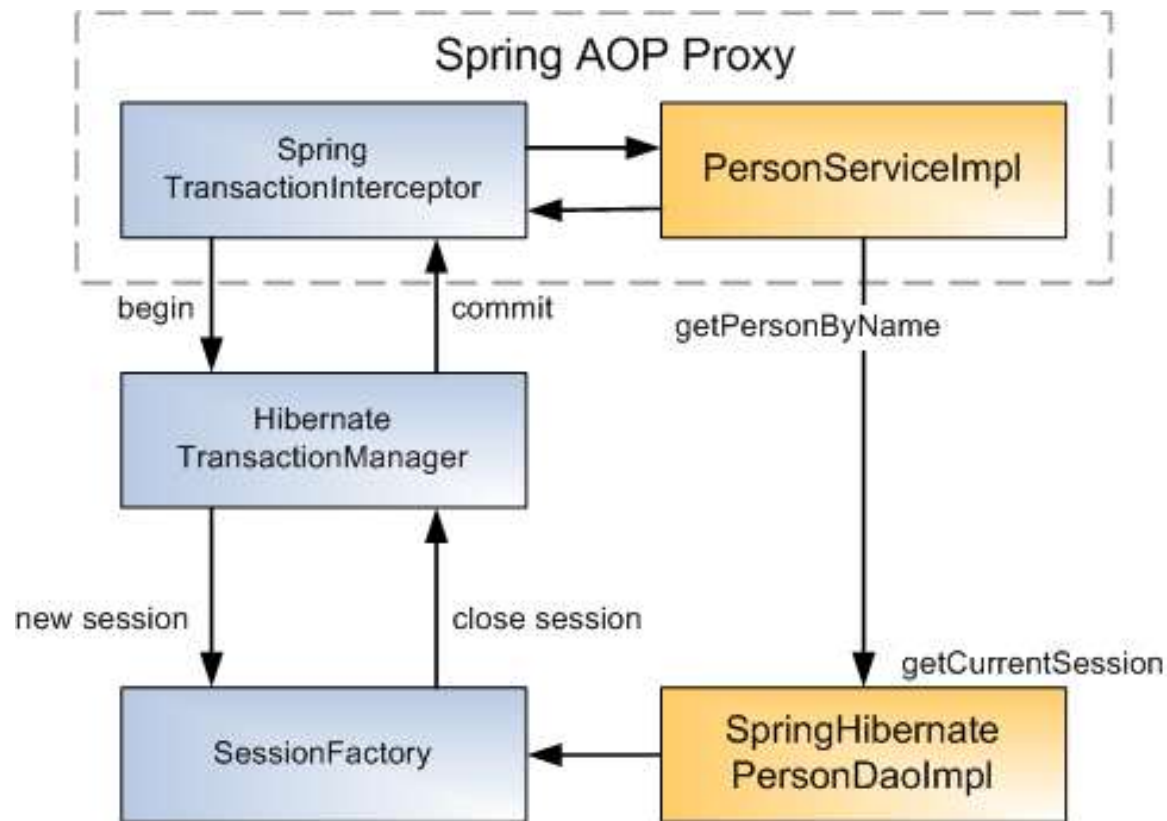
➤ Annotation du service pour la gestion transactionnelle

@Transactional

```
public interface IPersonService {...}
```

Configuration d'Hibernate avec Spring

➤ Principe de fonctionnement





Travaux pratiques

128

Réaliser les travaux pratiques suivants:

TP 12 : Spring et Hibernate (100% Hibernate)

TP 13 : Spring et JPA (100% JPA)